

Application Flows

Using a conversation to initialize the main flows of the target application:

<https://gemini.google.com/share/2d02d16c9862>

This is a Google Cloud-native stack. Using **Firebase Data Connect** is a strategic choice because it provides the developer experience of Firebase (GraphQL) with the capabilities of PostgreSQL (relational queries + vector search).

Here is the complete architecture overview tailored to your specific stack and requirements.

I. High-Level Architecture Diagram

The system follows a **Event-Driven & Microservices** pattern. The Frontend interacts directly with Firebase services for standard operations (Auth, Read Data, Upload Files) and calls Python Microservices for "Intelligent" operations (Chat, Ingestion).

II. Infrastructure & Services Breakdown

1. Frontend Layer

- **Framework:** Next.js (React).
- **Hosting:** Firebase Hosting (serves the static assets and Next.js SSR functions).
- **Responsibilities:**
 - User Interface (Teacher Dashboard, Student Chat, Topics Graph Visualization, Teacher Analytics).
 - Authentication handling via Firebase SDK.
 - Direct file uploads to Cloud Storage (using Firebase Client SDK).
 - GraphQL queries/mutations via Data Connect SDK.

2. Data Layer (The "State")

- **Relational & Vector Data: Firebase Data Connect.**
 - *Underlying Tech:* Cloud SQL (PostgreSQL).
 - *Why:* It handles our complex relational schema (Topics ↔ Chunks) and **Vector Search** (using the `pgvector` extension in Postgres) for the RAG pipeline.
 - *Access:* GraphQL API.
- **File Storage: Cloud Storage for Firebase.**
 - Stores raw files (PDF, PPTX, DOCX).
 - Stores processed assets (images extracted from slides).
 - Stores converted markdown files.

3. Backend Logic (Compute)

- **Runtime:** Google Cloud Run (Serverless Containers).
- **Framework:** Python **FastAPI**.
- **Communication:**
 - Exposes REST endpoints for the Frontend.
 - Receives **Eventarc** triggers (e.g., when a file is uploaded to Storage).

4. AI & Intelligence Layer

- **Framework: DSPy.**
 - Used inside the Python services to orchestrate LLM calls.
 - Manages "Signatures" (Input/Output definitions) and "Modules" (Chain of Thought, RAG).
 - **LLM Provider:** Vertex AI (Google Gemini models), OpenAI, or Qwen hosted on <https://openrouter.ai> accessed via DSPy.
-

III. Backend Microservices Design

To keep the system modular, we split the Python backend into two logical services (deployed as separate Cloud Run services).

Service A: The Ingestion Service (Async Worker)

- **Trigger:** Eventarc trigger (fires when a file is uploaded to Cloud Storage).
- **Core Responsibilities (US1 - US3):**
 1. **File Parsing:** Downloads file from Storage. Uses libraries like `MarkItDown` to convert raw files into markdown.
 2. **DSPy Extraction Module:**
 - *Signature:* `Context` -> `ExtractedTopics`, `Summary`
 - *Logic:* Analyzes text to find topics (matching against Seed Topics from US5).
 3. **Chunking:** Splits text semantically with libraries such as `chonkie.ai` <https://github.com/chonkie-inc/chonkie>
 4. **Embedding:** Generates vectors for chunks using an embedding model (e.g., `text-embedding-gecko`).
 5. **Persistence:** Writes structured data (Chunks, Topics, Vectors) into Postgres via the Data Connect GraphQL mutation (or direct SQL if performance demands it).
 6. **Status Update:** Updates the `SourceDocument` status to `ANALYZED`.

It could make sense to split this microservice into smaller components (ingest, convert, analyze).

Service B: The Tutor Service (Synchronous API)

- **Trigger:** HTTPS Requests from the Next.js Frontend.
 - **Core Responsibilities (US6):**
 1. **Student Memory Manager:**
 - Retrieves student history from Data Connect.
 - Constructs the "Memory Context" for the LLM.
 2. **DSPy RAG Module:**
 - *Signature:* StudentQuery, ChatHistory, Memory -> SocraticQuestion, StudentMemory
 - *Logic:* Performs vector search in Postgres, retrieves chunks, and generates the response. Updates the student memory as conversation proceeds.
 3. **Quiz Generator:**
 - *DSPy Module:* Topic, Difficulty -> QuizJSON
 4. **Logging:** Writes chat logs and analytics back to Data Connect. This logging is in addition to the StudentMemory management.
-

IV. Data Connect Schema (GraphQL Definition)

Since we are using Data Connect, our `.gql` schema is our database definition. Here is how we map the previous logic to Data Connect with Vector support.

```
# schema.gql

# 1. Users & Roles
type User @table {
  id: String!
  email: String!
  role: String! # 'TEACHER' or 'STUDENT'
}

# 2. Course Structure
type Course @table {
  id: UUID! @default(expr: "uuid_generate_v4()")
  title: String!
  description: String
  status: String! # 'DRAFT', 'PUBLISHED'
  teacher: User!
}

type Topic @table {
  id: UUID!
  course: Course!
  name: String!
```

```

    definition: String
    # Relationships
    prerequisites: [Topic] @relation(name: "TopicPrereqs")
}

# 3. Content & Vectors
type SourceDocument @table {
    id: UUID!
    course: Course!
    storagePath: String! # Path in Cloud Storage
    status: String!
}

type Chunk @table {
    id: UUID!
    document: SourceDocument!
    content: String!
    # Vector Embedding for Semantic Search
    # Data Connect supports pgvector mapping here
    embedding: Vector! @col(type: "vector(768)")
    topics: [Topic] @relation(name: "ChunkTopics")
}

# 4. Student Interaction
# Probably will need to structure the messages
# support retrieval with window size and time parameters.
type ChatSession @table {
    id: UUID!
    student: User!
    course: Course!
    messages: JSON! # Stores the conversation history
}

type StudentMemory @table {
    id: UUID!
    student: User!
    key: String!
    value: String!
    confidence: Float!
}

```

V. Local Development Environment (Emulator)

To meet your requirement of running everything locally, here is the setup:

1. **Frontend:** Run `next dev` (localhost:3000).
2. **Firestore Emulators:**

- Run `firebase emulators:start`.
- This spins up:
 - **Auth Emulator:** (localhost:9099)
 - **Storage Emulator:** (localhost:9199)
 - **Data Connect Emulator:** (Requires Docker). It spins up a local Postgres container tailored for Data Connect.

3. Backend (Python):

- You cannot "emulate" Cloud Run, but you can run FastAPI locally.
- Run `uvicorn main:app --reload --port 8000`.
- **Crucial Config:** Configure your local Python app to talk to the *Local Postgres* container (exposed by Data Connect) and *Local Storage Emulator* instead of the real Google Cloud.
- *Environment Variables:* Bash

```
FIRESTORE_EMULATOR_HOST="localhost:8080"
FIREBASE_STORAGE_EMULATOR_HOST="localhost:9199"
DB_HOST="localhost" # Data Connect Postgres
DB_PORT="5432"
```

VI. Summary of Flow Implementation

Feature	Frontend (Next.js)	Backend (Python/DSPy)	Data (Data Connect)
Upload	Uploads file to Storage Emulator.	Ignores (or polls Storage in local dev).	Stores file metadata.
Ingest	Shows "Processing" spinner.	Event Trigger: Downloads file, parses, DSPy extracts topics, embeds.	Stores Chunks & Vectors.
Graph	Fetches Topics via GraphQL. Renders D3.js/ReactFlow graph.	N/A	Returns relational data.
Chat	Sends user msg to FastAPI.	DSPy: Generates embedding for query, runs Vector Search on DB, generates Socratic reply.	Logs chat, updates Memory.

VII. Summary of Frontend Pages

- Sign-up / Sign-in
- User Profile (name, role, courses, usage)
- Course Definition (course name, participants, schedule, teachers)
- Course Material Review (raw material, markdown, chunks, topics)
- Conversation (question, history, quizzes, skills)

- Student Memory Review (skills, <https://app.mem0.ai/dashboard/memories> example)
- Teacher Analytics (summary information per student, per topic)